

Face Mask Detection Using CNN

Supervised by : Prof. Dr. Gamal Behery

Team Members



Haneen Adel Abdelrahman Attia



Nourhan Mahmoud El Sherbiny



Menna Tarek Abdelrahman Elfar



Shahd Mohamed Ismail Hatab



Nada Yousry Ibrahim Elalfy



Hager Mohamed Elsayed Elazzazy

Introduction

Face Mask Detection is a Deep Learning project that uses Artificial Intelligence to identify whether a person is wearing a face mask or not from an image or video stream.

The project is based on Convolutional Neural Networks (CNN), which are widely used in Computer Vision applications. The model learns facial features from images and classifies them into two categories:

- With Mask
- Without Mask

This system can be integrated into smart surveillance cameras and real-time monitoring systems.



Problem Definition :

During the COVID-19 pandemic, wearing face masks became an essential safety measure to reduce the spread of infectious diseases. However, monitoring mask compliance manually in crowded places is difficult and time-consuming.

The problem addressed in this project is:
“Can Artificial Intelligence automatically detect whether a person is wearing a face mask correctly or not?”

To solve this problem, a CNN-based image classification model is developed to analyze facial images and predict mask usage automatically.

Importance of the Project

This project is important because it provides an automated and efficient solution for public safety monitoring

The project helps in:

- Reducing manual monitoring effort
- Improving public health safety
- Supporting smart city applications
- Detecting mask violations in real time
- Increasing the use of AI in healthcare systems

Real-world applications:

- Hospitals
- Airports
- Schools and universities
- Shopping malls
- Offices and companies
- Public transportation stations

The project also demonstrates the practical use of Deep Learning and Computer Vision technologies.

Prediction Goal

The main goal of the project is to predict whether a person in an image is:

- Wearing a face mask
- Not wearing a face mask

This is considered a Binary Classification problem because the model predicts one of two classes.

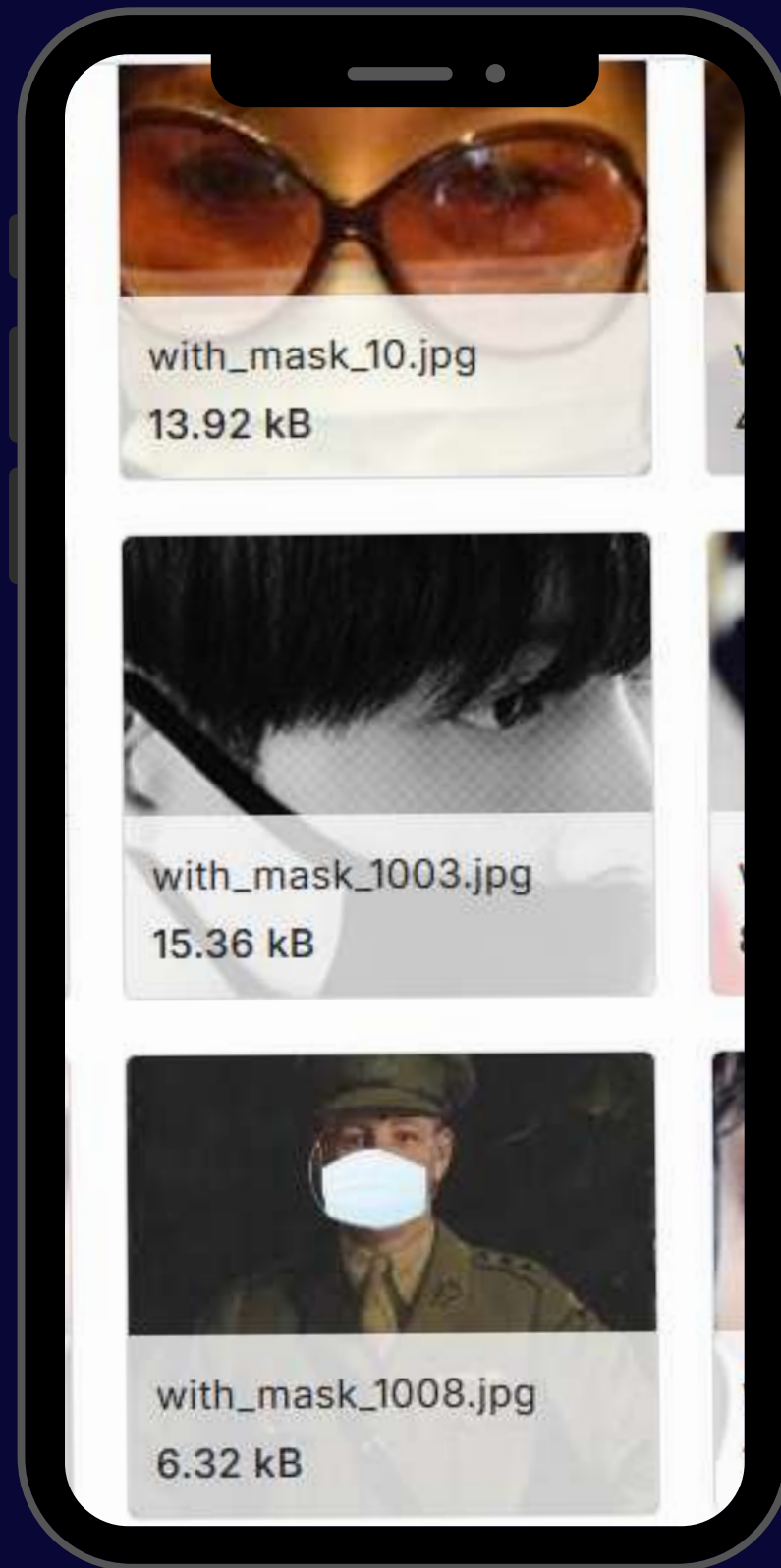
Input:

- Facial image

Output:

- Mask
- No Mask

The trained CNN model learns patterns and features from images to make accurate predictions.



Dataset Description

- The dataset used in this project was taken from Kaggle.
- It contains facial images of people:
 - Wearing face masks
 - Not wearing face masks
- [Face Mask Dataset on Kaggle](#)

Type of Data

- Image Classification Dataset
- RGB facial images
- JPG and PNG image formats

Dataset Size

- The dataset contains more than 7,500 facial images in total.
- Approximately 3,700 images are labeled as With Mask.
- Approximately 3,800 images are labeled as Without Mask.

Why This Dataset Is Suitable

This dataset is suitable for the project because:

- It contains a large number of images
- Images are already categorized
- It supports binary image classification
- It is commonly used in Computer Vision projects
- It helps train CNN models effectively

The dataset also contains images with:

- Different face angles
- Different lighting conditions
- Different mask styles
- Different backgrounds

This helps the model generalize better on real-world images.

Project Workflow

- Import Libraries
- Setting up path for training data
- Data Loading for training
- Data Explorations
- Print classes present in the data
- Create Models
- Train Model
- Finding LR
- Finetuning HyperParameter
- Interpret the results
- Prediction Using Trained Model
- Save and Load Model

Import Libraries

```
from fastai.vision.all import *  
from fastai.metrics import error_rate  
from pathlib import Path  
import os  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt
```

Setting up path for training data

```
x = r"C:\Users\rehan\Desktop\image project\Face Mask Dataset"  
path = Path(x)  
path.ls()
```

Data Loading For training

```
path = Path(r"C:\Users\rehan\Desktop\image project\Face Mask Dataset")

data = ImageDataLoaders.from_folder(
    path,
    train='Train',
    valid='Validation',
    item_tfms=Resize(224),
    bs=32
)
```

Data Explorations

Our image dataset is stored as .jpg files in 2 different folders, with each folder bearing the name of model of the images contained in the folder. We use the `ImageDataBunch.from_folder()` function to load the images and assign labels the images based on the name of the folder they're read from.

```
data.show_batch(max_n = 9 , figsize=(6,6))
```

Print Classes present in the data

- `print(data.vocab)` — Prints the names of the classes present in the dataset (With Mask / Without Mask).
- `len(data.vocab)` — Shows the total number of unique classes found in the data.
- `data.c` — Confirms the number of output channels/classes for the model

```
print(data.vocab)
len(data.vocab)
data.c
```

Create Models

In this project, we used a Convolutional Neural Network for image classification. However, instead of building and training a model from scratch, we applied a more powerful technique called Transfer Learning.

We used a pre-trained ResNet model that was already trained on the ImageNet dataset, which contains around one million images. This means the model has already learned important visual features such as edges, shapes, and textures.

We then adapted this model to our specific task by freezing most of its layers and training only the final layer. This allows the model to reuse existing knowledge while learning our dataset efficiently.

We chose this approach because our dataset is relatively small, with only around 4000 images, and training a deep model from scratch would require a much larger dataset. Finally, the model outputs two classes: With Mask and Without Mask.

Create Models code

```
1 learn = cnn_learner(data, resnet50, metrics=accuracy )
```

✓ 0.8s

Finding LR

In this step, we focused on finding the best learning rate for training the model. The learning rate is an important parameter that controls how quickly the model updates its weights during training.

If the learning rate is too high, the model may become unstable and fail to converge.

If it is too low, the training process becomes very slow and inefficient.

So, we tested different learning rates to choose the most suitable one that gives stable learning and good performance.

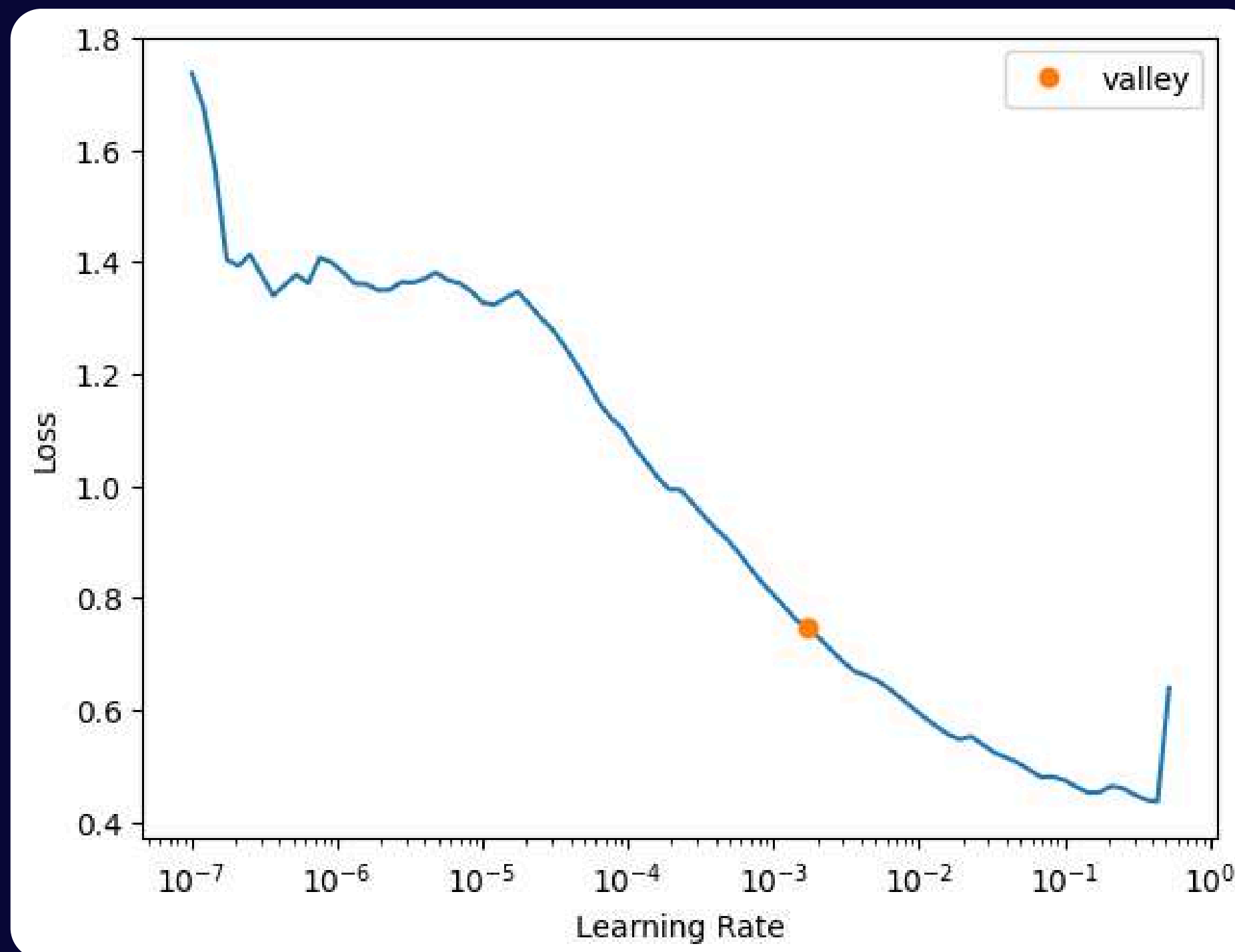
The selected learning rate was then used to train the final model and improve its accuracy

Finding LR Code

```
1 learn.lr_find()
```

✓ 6m 43.2s

Finding LR output





Train Model

In this step, we trained the model using the prepared training dataset.

The model learns patterns and features from the images of faces with and without masks.

During training, the model continuously adjusts its internal weights to reduce the error between predicted and actual results.

We used the Adam optimizer and Binary Crossentropy as the loss function because this is a binary classification problem.

The model was trained over multiple epochs, and with each epoch, its performance gradually improved, leading to better accuracy in classification.

Train Model code

```
1 lr1 = 1e-3
2 learn.fit_one_cycle(4,slice(lr1))
```

✓ 99m 51.7s

epoch	train_loss	valid_loss	accuracy	time
0	0.040386	0.015414	0.991250	23:54
1	0.026967	0.017034	0.993750	24:06
2	0.020548	0.010348	0.996250	26:31
3	0.019667	0.007192	0.997500	25:19

Hyper Parameter Tuning

In this step, we performed hyperparameter tuning.

Hyperparameters are settings that control how the model is trained, but they are not learned from the data. They are defined before training starts.

In this project, we tuned important parameters such as the learning rate, number of epochs, and batch size.

We tested different values for these parameters to find the best combination that gives the highest accuracy and stable training performance.

This step helps improve the overall performance of the model and makes the training process more efficiency

Hyper Parameter Tuning code

```
1 learn.unfreeze()  
2 learn.fit_one_cycle(2,slice(1e-5,1e-3))
```

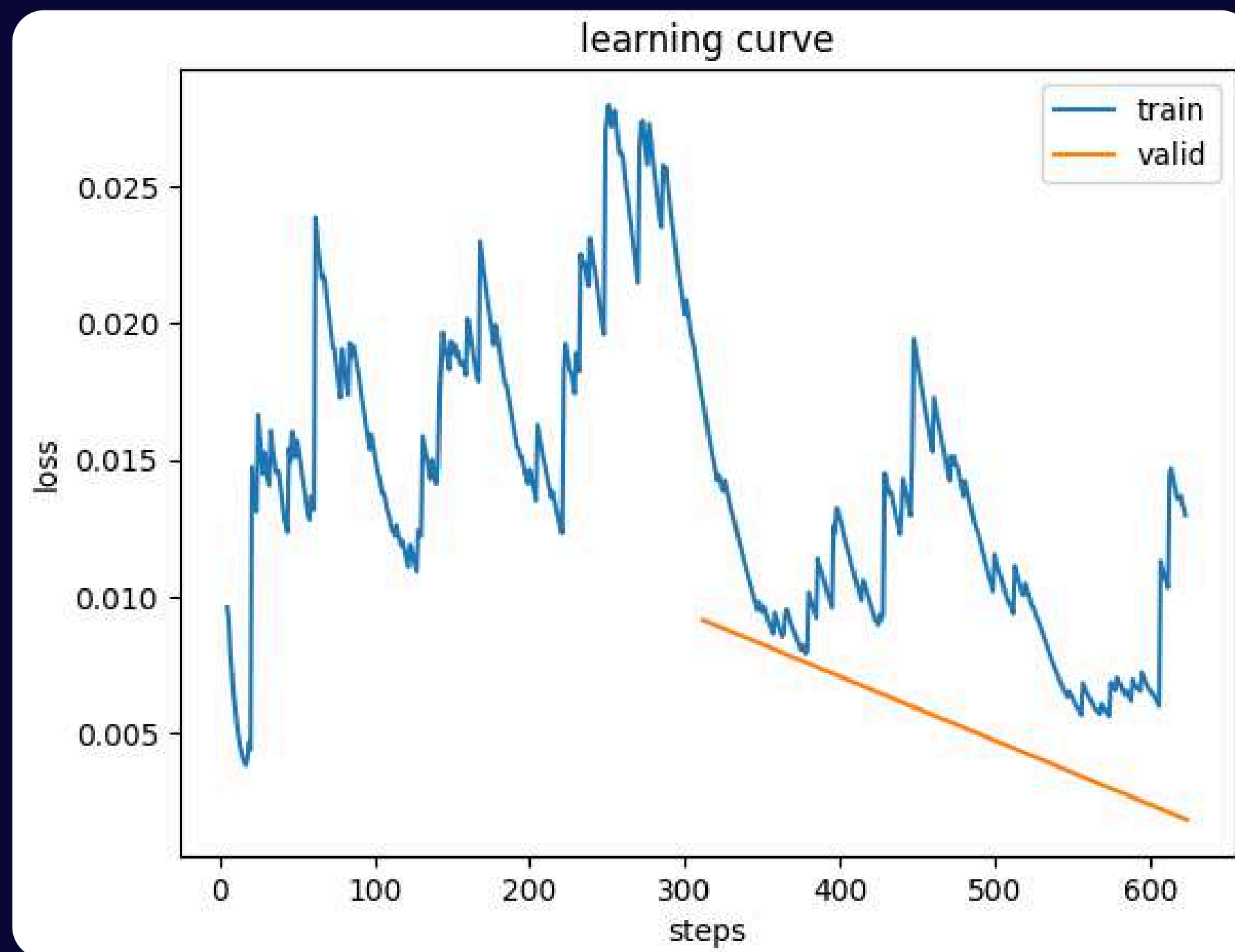
✓ 72m 10.2s

epoch	train_loss	valid_loss	accuracy	time
0	0.017365	0.009146	0.998750	36:09
1	0.012975	0.001850	1.000000	36:01

Hyper Parameter Tuning Output

```
1 learn.recorder.plot_loss()
```

✓ 0.1s

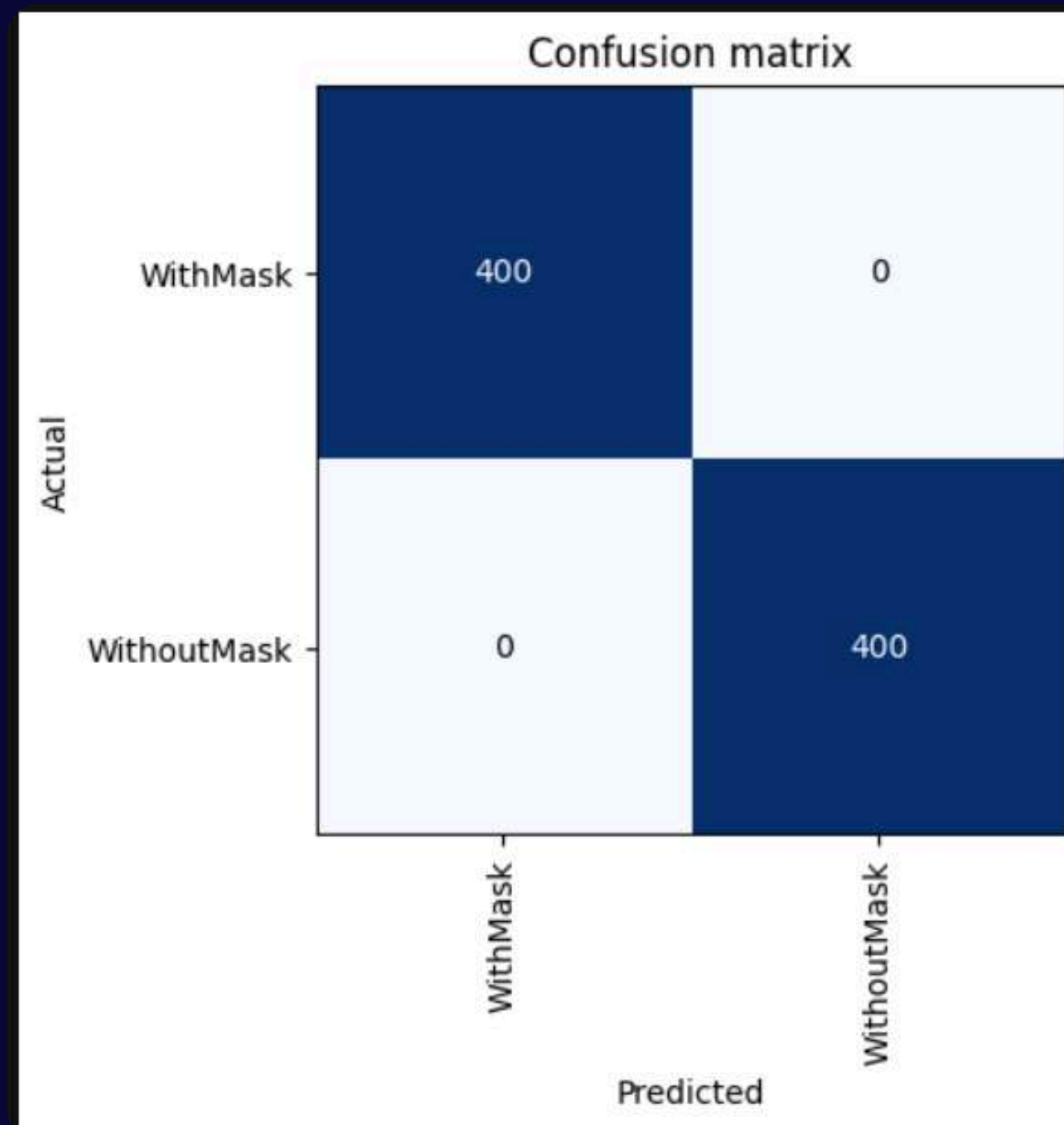


Interpret the results code

```
1 interp = ClassificationInterpretation.from_learner(learn)
2 interp.plot_confusion_matrix()
```

✓ 1m 38.0s

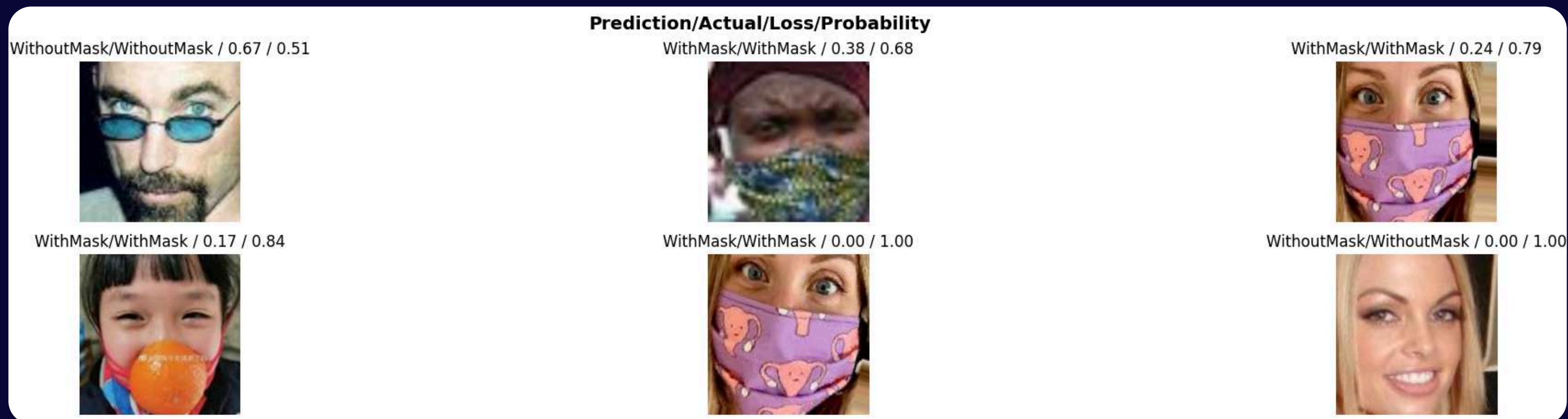
confusion_matrix



Interpret the results output

```
1 interp.plot_top_losses(6,figsize = (25,5))
```

✓ 0.8s



Performance Evaluation

```
1 # ===== Validation Accuracy =====
2 learn.dls = ImageDataLoaders.from_folder(
3     path,
4     train='Train',
5     valid='Validation',
6     item_tfms=Resize(224),
7     bs=32
8 )
9
10 val_loss, val_acc = learn.validate()
11
12 # ===== Test Accuracy =====
13 learn.dls = ImageDataLoaders.from_folder(
14     path,
15     train='Train',
16     valid='Test',
17     item_tfms=Resize(224),
18     bs=32
19 )
20
21 test_loss, test_acc = learn.validate()
22
23 # ===== Print Results =====
24 print("===== Model Performance =====")
25 print(f"Validation Loss: {val_loss:.4f}")
26 print(f"Validation Accuracy: {val_acc:.4f}")
27
28 print(f"Test Loss: {test_loss:.4f}")
29 print(f"Test Accuracy: {test_acc:.4f}")
```

✓ 2m 3.7s

```
===== Model Performance =====
Validation Loss: 0.1035
Validation Accuracy: 0.9762
Test Loss: 0.0926
Test Accuracy: 0.9657
```



THANK YOU